University of Stirling

ITNPBD2 Representing and Manipulating Data

Assignment Autumn 2025

A Consultancy Job for JC Penney

This notebook forms the assignment instructions and submission document of the assignment for ITNPBD2. Read the instructions carefully and enter code into the cells as indicated.

You will need these five files, which were in the Zip file you downloaded from the course webpage:

- jcpenney_reviewers.json
- jcpenney_products.json
- products.csv
- reviews.csv
- users.csv

The data in these files describes products that have been sold by the American retail giant, JC Penney, and reviews by customers who bought them. Note that the product data is real, but the customer data is synthetic.

Your job is to process the data, as requested in the instructions in the markdown cells in this notebook.

Completing the Assignment

Rename this file to be xxxxxx_BD2 where xxxxxx is your student number, then type your code and narrative description into the boxes provided. Add as many code and markdown cells as you need. The cells should contain:

- **Text narrative describing what you did with the data**
- **The code that performs the task you have described**
- **Comments that explain your code**

The final structure (in PDF) of your report must:

- **Start from the main insights observed (max 5 pages)**
- **Include as an appendix the source code used for producing those insights (max 15 pages)**
- **Include an AI cover sheet (provided on Canvas), which must contain a link to a versioned notebook file in OneDrive or another platform for version checks.**

Marking Scheme

The assessment will be marked against the university Common Marking Scheme (CMS)

Here is a summary of what you need to achieve to gain a grade in the major grade bands:

Grade	Requirement
Fail	You will fail if your code does not run or does not achieve even the basics of the task. You may also fail if you submit code without either comments or a text explanation of what the code does.
Pass	To pass, you must submit sufficient working code to show that you have mastered the basics of the task, even if not everything works completely. You must include some justifications for your choice of methods, but without mentioning alternatives.
Merit	For a merit, your code must be mostly correct, with only small problems or parts missing, and your comments must be useful rather than simply re-stating the code in English. Most choices for methods and structures should be explained and alternatives mentioned.
Distinction	For a distinction, your code must be working, correct, and well commented and shows an appreciation of style, efficiency and reliability. All choices for methods and structures are concisely justified and alternatives are given well thought considerations. For a distinction, your work should be good enough to present to executives at the company.

The full details of the CMS can be found here

https://www.stir.ac.uk/about/professional-services/student-academic-and-corporate-services/academic-registry/academic-policy-and-practice/quality-handbook/assessment-policy-and-procedure/appendix-2-postgraduate-common-marking-scheme/

Note that this means there are not certain numbers of marks allocated to each stage of the assignment. Your grade will reflect how well your solutions and comments demonstrate that you have achieved the learning outcomes of the task.

Submission

When you are ready to submit, **print** your notebook as PDF (go to File -> Print Preview) in the Jupyter menu. Make sure you have run all the cells and that their output is displayed. Any lines of code or comments that are not visible in the pdf should be broken across several lines. You can then submit the file online.

Late penalties will apply at a rate of three marks per day, up to a maximum of 7 days. After 7 days you will be given a mark of 0. Extensions will be considered under acceptable circumstances outside your control.

Academic Integrity

This is an individual assignment, and so all submitted work must be fully your own work.

The University of Stirling is committed to protecting the quality and standards of its awards. Consequently, the University seeks to promote and nurture academic integrity, support staff academic integrity, and support students to understand and develop good academic skills that facilitate academic integrity.

In addition, the University deals decisively with all forms of Academic Misconduct.

Where a student does not act with academic integrity, their work or behaviour may demonstrate Poor Academic Practice or it may represent Academic Misconduct.

Poor Academic Practice

Poor Academic Practice is defined as: "The submission of any type of assessment with a lack of referencing or inadequate referencing which does not effectively acknowledge the origin of words, ideas, images, tables, diagrams, maps, code, sound and any other sources used in the assessment."

Academic Misconduct

Academic Misconduct is defined as: "any act or attempted act that does not demonstrate academic integrity and that may result in creating an unfair academic advantage for you or another person, or an academic disadvantage for any other member or member of the academic community."

Plagiarism is presenting somebody else's work as your own **and includes the use of artificial intelligence tools beyond AIAS Level 2 or the use of Large Language Models.**. Plagiarism is a form of academic misconduct and is taken very seriously by the University. Students found to have plagiarised work can have marks deducted and, in serious cases, even be expelled from the University. Do not submit any work that is not entirely your own. Do not collaborate with or get help from anybody else with this assignment.

The University of Stirling's full policy on Academic Integrity can be found at:

https://www.stir.ac.uk/about/professional-services/student-academic-and-corporate-services/academic-registry/academic-policy-and-practice/quality-handbook/academic-integrity-policy-and-academic-misconduct-procedure/

The Assignment

Your task with this assignment is to use the data provided to demonstrate your Python data manipulation skills.

There are three `.csv` files and two `.json` files so you can process different types of data. The files also contain unstructured data in the form of natural language in English and links to images that you can access from the JC Penney website (use the field called `product_image_urls`).

Start with easy tasks to show you can read in a file, create some variables and data structures, and manipulate their contents. Then move onto something more interesting.

Look at the data that we provided with this assessment and think of something interesting to do with it using whatever libraries you like. Describe what you decide to do with the data and why it might be interesting or useful to the company to do it.

You can add additional data if you need to - either download it or access it using `requests`. Produce working code to implement your ideas in as many cells as you need below. There is no single right answer, the aim is to simply show you are competent in using python for data analysis. Exactly how you do that is up to you.

For a distinction class grade, this must show originality, creative thinking, and insights beyond what you've been taught directly on the module.

Structure

You may structure the appendix of the project how you wish, but here is a suggested guideline to help you organise your work, based on the CRISP-DM data science methodology:

1. **Business understanding** - What business context is the data coming from? What insights would be valuable in that context, and what data would be required for that purporse?
2. **Data understanding and preparation** - Explore the data and show you understand its structure and relations, with the aid of appropriate visualisation techniques. Assess the data quality, which insights you would be able to answer from it, and what preparation the data would require. Add new data from another source if required to bring new insights to the data you already have.

3. **Data modeling (optional)** - Would modeling be required for the insights you have considered? Use appropriate techniques, if so.
4. **Evaluation and deployment** - How do the insights you obtained help the company, and how can should they be adopted in their business? If modeling techniques have been adopted, are their use scientifically sound and how should they be mantained?

Remember to make sure you are working completely on your own.

Don't work in a group or with a friend

In [2]:
```
# Put your code and comments in cells below here
# Add as many cells as you need
```

Step 1: Data Loading and Initial Inspection

The first section of this code will include loading the data and verifying the structure for future use.

In this step:

- Two json files and the three csv files were loaded.
- A dictionary was used to manage the csvs effectively.
- The record counts were printed.
- The first ten rows of all data were then printed to gain an understanding of structure and content.

This step ensures all data is loaded and visible and some intial information on the dataset is ascertained.

In [3]:
```
# Load in JCpenny reviewer and product json files
import pandas as pd

products_json_df = pd.read_json("jcpenney_products.json", lines=True)
reviewers_json_df = pd.read_json("jcpenney_reviewers.json", lines=True)

# Preview the structure and confirm row counts

display(products_json_df.head())
display(reviewers_json_df.head())

print("Products JSON rows:", len(products_json_df))
print("Reviewers JSON rows:", len(reviewers_json_df))

# Load CSV data and display row counts
csv_files = {
```

```python
    "Products": pd.read_csv("products.csv"),
    "Reviews": pd.read_csv("reviews.csv"),
    "Users": pd.read_csv("users.csv")
}

for name, df in csv_files.items():
    print(f"{name}: {len(df)} rows")

# Unpack DataFrames for later use
products_csv_df, reviews_csv_df, users_csv_df = csv_files.values()

# Preview the first 10 rows of each CSV to inspect structure
for name, df in csv_files.items():
    print(f"\n{name} Preview:")
    display(df.head(10))
```

	uniq_id	sku	name_title	description	list_price	sale
0	b6c0b6bea69c722939585baeac73c13d	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09	
1	93e5272c51d8cce02597e3ce67b7ad0a	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09	
2	013e320f2f2ec0cf5b3ff5418d688528	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09	
3	505e6633d81f2cb7400c0cfa0394c427	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09	
4	d969a8542122e1331e304b09f81a83f6	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09	

	Username	DOB	State	Reviewed
0	bkpn1412	31.07.1983	Oregon	[cea76118f6a9110a893de2b7654319c0]
1	gqjs4414	27.07.1998	Massachusetts	[fa04fe6c0dd5189f54fe600838da43d3]
2	eehe1434	08.08.1950	Idaho	[]
3	hkxj1334	03.08.1969	Florida	[f129b1803f447c2b1ce43508fb822810, 3b0c9bc0be6...
4	jjbd1412	26.07.2001	Georgia	[]

```
Products JSON rows: 7982
Reviewers JSON rows: 5000
Products: 7982 rows
Reviews: 39063 rows
Users: 5000 rows

Products Preview:
```

	Uniq_id	SKU	Name	Description	Price	Av_Score
0	b6c0b6bea69c722939585baeac73c13d	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	2.625
1	93e5272c51d8cce02597e3ce67b7ad0a	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	3.000
2	013e320f2f2ec0cf5b3ff5418d688528	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	2.625
3	505e6633d81f2cb7400c0cfa0394c427	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	3.500
4	d969a8542122e1331e304b09f81a83f6	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	3.125
5	cf73bb2bd93bbd6e1bdf48d399992270	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	3.000
6	8ffd0ef4fcaf1a82fb514aba5d20e05b	pp5006790247	Alfred Dunner® Feels Like Spring 3/4 Sleeve Le...	For easygoing style youll love, wear our stret...	65.27	3.750
7	4d9337e3c8f974d3c420cdc5c58b3fc3	pp5007090172	Alfred Dunner® Feels Like	Spring is in the air with our 3/4-	NaN	1.000

	Uniq_id	SKU	Name	Description	Price	Av_Score
			Spring 3/4-Sleeve Le...	sleeve leaf ...		
8	44f8f8f108c6856acf9630dd1d78516d	pp5007080134	Alfred Dunner® Feels Like Spring 3/4-Sleeve Wa...	NaN	NaN	4.500
9	8d1d057f5f808c10ce243c222ab0ef6e	pp5007080134	Alfred Dunner® Feels Like Spring 3/4-Sleeve Wa...	NaN	NaN	2.500

```
Reviews Preview:
```

	Uniq_id	Username	Score	Review
0	b6c0b6bea69c722939585baeac73c13d	fsdv4141	2	You never have to worry about the fit...Alfred...
1	b6c0b6bea69c722939585baeac73c13d	krpz1113	1	Good quality fabric. Perfect fit. Washed very ...
2	b6c0b6bea69c722939585baeac73c13d	mbmg3241	2	I do not normally wear pants or capris that ha...
3	b6c0b6bea69c722939585baeac73c13d	zeqg1222	0	I love these capris! They fit true to size and...
4	b6c0b6bea69c722939585baeac73c13d	nvfn3212	3	This product is very comfortable and the fabri...
5	b6c0b6bea69c722939585baeac73c13d	aajh3423	2	I did not like the fabric. It is 100% polyeste...
6	b6c0b6bea69c722939585baeac73c13d	usvp2142	1	What a great deal. Beautiful Pants. Its more t...
7	b6c0b6bea69c722939585baeac73c13d	yemw3321	1	Alfred Dunner has great pants, good fit and ve...
8	93e5272c51d8cce02597e3ce67b7ad0a	tpcu2211	3	You never have to worry about the fit...Alfred...
9	93e5272c51d8cce02597e3ce67b7ad0a	vutl2421	2	Good quality fabric. Perfect fit. Washed very ...

```
Users Preview:
```

	Username	DOB	State
0	bkpn1412	31.07.1983	Oregon
1	gqjs4414	27.07.1998	Massachusetts
2	eehe1434	08.08.1950	Idaho
3	hkxj1334	03.08.1969	Florida
4	jjbd1412	26.07.2001	Georgia
5	rriz2231	08.08.1951	Montana
6	dvmv4241	02.08.1975	Pennsylvania
7	ibtk1124	31.07.1983	Connecticut
8	ryqz4413	28.07.1992	Arkansas
9	ezng1232	03.08.1968	Nebraska

Step 2: Data Understanding and Quality Check

Before cleaning and merging, the datasets are explored to assess the structure and quality.

In this step:

- Column names are reviewed for each dataset.
- Missing values are identified and summarised.
- A visual heatmap highlights patterns of missing data.

This step ensures that any inconsistencies or incomplete records can be addressed before further analysis.

In [4]:
```
# Step 2: Data Understanding and Quality Check

# Creating a dictionary to store all data frames
all_datasets = {
    "Products CSV": products_csv_df,
    "Reviews CSV": reviews_csv_df,
    "Users CSV": users_csv_df,
    "Products JSON": products_json_df,
    "Reviewers JSON": reviewers_json_df
}

# Loop through and print the column names for each dataset
for name, df in all_datasets.items():
    print(f"\n{name} Columns ({len(df.columns)} total):")
    print(df.columns.tolist())

# Missing Values Summary
print("\nMissing values in Products CSV:")
print(products_csv_df.isna().sum())
```

```python
print("\nMissing values in Reviews CSV:")
print(reviews_csv_df.isna().sum())

print("\nMissing values in Users CSV:")
print(users_csv_df.isna().sum())

print("\nMissing values in jcpenney_products.json:")
print(products_json_df.isna().sum().sort_values(ascending=False))

print("\nMissing values in jcpenney_reviewers.json:")
print(reviewers_json_df.isna().sum().sort_values(ascending=False))

# Visualise missing data for the identified data set with data missing
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 4))
sns.heatmap(products_csv_df.isna(), cbar=False, cmap="coolwarm")
plt.title("Missing Values in Products CSV")
plt.show()
```

```
Products CSV Columns (6 total):
['Uniq_id', 'SKU', 'Name', 'Description', 'Price', 'Av_Score']

Reviews CSV Columns (4 total):
['Uniq_id', 'Username', 'Score', 'Review']

Users CSV Columns (3 total):
['Username', 'DOB', 'State']

Products JSON Columns (15 total):
['uniq_id', 'sku', 'name_title', 'description', 'list_price', 'sale_price', 'categor
y', 'category_tree', 'average_product_rating', 'product_url', 'product_image_urls',
'brand', 'total_number_reviews', 'Reviews', 'Bought With']

Reviewers JSON Columns (4 total):
['Username', 'DOB', 'State', 'Reviewed']

Missing values in Products CSV:
Uniq_id           0
SKU              67
Name              0
Description     543
Price          2166
Av_Score          0
dtype: int64

Missing values in Reviews CSV:
Uniq_id     0
Username    0
Score       0
Review      0
dtype: int64

Missing values in Users CSV:
Username    0
DOB         0
State       0
dtype: int64

Missing values in jcpenney_products.json:
uniq_id                   0
sku                       0
name_title                0
description               0
list_price                0
sale_price                0
category                  0
category_tree             0
average_product_rating    0
product_url               0
product_image_urls        0
brand                     0
total_number_reviews      0
Reviews                   0
Bought With               0
dtype: int64
```

```
Missing values in jcpenney_reviewers.json:
Username    0
DOB         0
State       0
Reviewed    0
dtype: int64
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[4], line 34
     31 print(reviewers_json_df.isna().sum().sort_values(ascending=False))
     33 # Visualise missing data for the identified data set with data missing
---> 34 import seaborn as sns
     35 import matplotlib.pyplot as plt
     37 plt.figure(figsize=(10, 4))

ModuleNotFoundError: No module named 'seaborn'
```

Step 3: Data Cleaning and Preperation After assessing the data structure and quality, the next step is to clean and standardise it to ensure consistency across files before merging.

In this step:

- All collumn names were standardised by converting to lowercase and key fields renamed.
- All missing values were flagged
- DOB was changed into age groups for more effective analysis
- Removed duplicates
- Prepare the data for merging by ensuring join keys were consistant

In [5]:

```
# STEP 3 – Clean and Prepare Data for Analysis
import pandas as pd
import numpy as np
import re

# Standardise column names
def standardise_columns(df):
    df.columns = df.columns.str.strip().str.lower()
    return df

datasets = [products_csv_df, reviews_csv_df, users_csv_df, products_json_df, review
datasets = [standardise_columns(df) for df in datasets]
products_csv_df, reviews_csv_df, users_csv_df, products_json_df, reviewers_json_df

# Rename key identifier columns for consistency
rename_map = {'uniq_id': 'product_id', 'username': 'user_id'}
for df in [products_csv_df, reviews_csv_df, users_csv_df, products_json_df, reviewe
    df.rename(columns={k: v for k, v in rename_map.items() if k in df.columns}, inp

# Clean product pricing information and extract numeric values from list_price and
for col in ['list_price', 'sale_price']:
    if col in products_json_df.columns:
        products_json_df[col] = (
```

```
            products_json_df[col]
            .astype(str)
            .str.extract(r'(\d+\.?\d*)')[0]  # Extract numeric portions like 41.09
            .astype(float)
        )

# Flag missing price values if either list or sale price is missing
products_json_df['price_missing_flag'] = (
    products_json_df[['list_price', 'sale_price']].isna().any(axis=1).astype(int)
)

# Create a discount flag and percentage (only when both prices exist)
if {'list_price', 'sale_price'}.issubset(products_json_df.columns):
    products_json_df['discount_flag'] = (
        (products_json_df['sale_price'] < products_json_df['list_price'])
        & products_json_df['sale_price'].notna()
        & products_json_df['list_price'].notna()
    ).astype(int)

# Discount percentage (only for valid discount cases)
    products_json_df['discount_percent'] = np.where(
        products_json_df['discount_flag'] == 1,
        ((products_json_df['list_price'] - products_json_df['sale_price']) /
         products_json_df['list_price'] * 100).round(2),
        np.nan
    )

# Clean user data
if 'dob' in users_csv_df.columns:
    # Handle UK date format (dd.mm.yyyy)
    users_csv_df['dob'] = pd.to_datetime(users_csv_df['dob'], format='%d.%m.%Y', er

    # Calculate age in years
    users_csv_df['age'] = ((pd.Timestamp('today') - users_csv_df['dob']).dt.days /

    # Create age groups
    bins = [0, 25, 40, 60, 100]
    labels = ['<25', '25-40', '40-60', '60+']
    users_csv_df['age_group'] = pd.cut(users_csv_df['age'], bins=bins, labels=label

# Validate cleaning results

print(f"Products missing any price: {products_json_df['price_missing_flag'].sum()}"
print(f"Products with discounts: {products_json_df['discount_flag'].sum()}")
print(f"Users with missing DOBs: {users_csv_df['dob'].isna().sum()}")

# --- 6. Preview cleaned data ---
display(products_json_df[['product_id', 'list_price', 'sale_price',
                          'discount_flag', 'discount_percent', 'price_missing_flag'

display(users_csv_df[['user_id', 'dob', 'age', 'age_group']].head(10))
```

```
Products missing any price: 2166
Products with discounts: 5772
Users with missing DOBs: 0
```

	product_id	list_price	sale_price	discount_flag	discount_percent
0	b6c0b6bea69c722939585baeac73c13d	41.09	24.1600	1	41.2
1	93e5272c51d8cce02597e3ce67b7ad0a	41.09	24.1600	1	41.2
2	013e320f2f2ec0cf5b3ff5418d688528	41.09	24.1600	1	41.2
3	505e6633d81f2cb7400c0cfa0394c427	41.09	24.1600	1	41.2
4	d969a8542122e1331e304b09f81a83f6	41.09	24.1600	1	41.2
5	cf73bb2bd93bbd6e1bdf48d399992270	41.09	24.1600	1	41.2
6	8ffd0ef4fcaf1a82fb514aba5d20e05b	65.27	39.1600	1	40.0
7	4d9337e3c8f974d3c420cdc5c58b3fc3	NaN	70.1000	0	NaN
8	44f8f8f108c6856acf9630dd1d78516d	NaN	62.8574	0	NaN
9	8d1d057f5f808c10ce243c222ab0ef6e	NaN	62.8574	0	NaN

	user_id	dob	age	age_group
0	bkpn1412	1983-07-31	42.3	40–60
1	gqjs4414	1998-07-27	27.3	25–40
2	eehe1434	1950-08-08	75.2	60+
3	hkxj1334	1969-08-03	56.3	40–60
4	jjbd1412	2001-07-26	24.3	<25
5	rriz2231	1951-08-08	74.2	60+
6	dvmv4241	1975-08-02	50.3	40–60
7	ibtk1124	1983-07-31	42.3	40–60
8	ryqz4413	1992-07-28	33.3	25–40
9	ezng1232	1968-08-03	57.3	40–60

In [6]:

```python
import re
import numpy as np
import pandas as pd

# --- Clean up badly formatted prices ---
for col in ['list_price', 'sale_price']:
    if col in products_json_df.columns:
        products_json_df[col] = (
            products_json_df[col]
            .astype(str)
            .str.replace(r'[\$,]', '', regex=True)   # remove $ and commas
            .str.extract(r'(\d+\.\d+)')[0]           # extract first valid decimal
```

```
            .astype(float)
        )
        # Fix unrealistic large prices (e.g., 5000+ likely parsing errors)
        products_json_df.loc[products_json_df[col] > 1000, col] = np.nan

# Flag missing prices
products_json_df['price_missing_flag'] = (
    products_json_df[['list_price', 'sale_price']].isna().any(axis=1).astype(int)
)

# Quick summary check
print(products_json_df[['list_price', 'sale_price']].describe())
display(products_json_df[['product_id', 'list_price', 'sale_price']].head(10))
```

```
        list_price   sale_price
count  5662.000000  7837.000000
mean     87.070394    64.242529
std     112.732672   107.192577
min       8.010000     3.450000
25%      40.390000    22.450000
50%      57.030000    34.910000
75%      84.600000    59.540000
max     996.940000   999.150000
```

	product_id	list_price	sale_price
0	b6c0b6bea69c722939585baeac73c13d	41.09	24.1600
1	93e5272c51d8cce02597e3ce67b7ad0a	41.09	24.1600
2	013e320f2f2ec0cf5b3ff5418d688528	41.09	24.1600
3	505e6633d81f2cb7400c0cfa0394c427	41.09	24.1600
4	d969a8542122e1331e304b09f81a83f6	41.09	24.1600
5	cf73bb2bd93bbd6e1bdf48d399992270	41.09	24.1600
6	8ffd0ef4fcaf1a82fb514aba5d20e05b	65.27	39.1600
7	4d9337e3c8f974d3c420cdc5c58b3fc3	NaN	70.1000
8	44f8f8f108c6856acf9630dd1d78516d	NaN	62.8574
9	8d1d057f5f808c10ce243c222ab0ef6e	NaN	62.8574

Step 4: Data Analysis and Visualisation

In [12]:
```
# STEP 4 — Clean and Enhance Reviews Data
!pip install seaborn matplotlib nltk
import seaborn as sns
import matplotlib.pyplot as plt
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk
import numpy as np
import pandas as pd

# Make sure the sentiment model is available
```

```python
nltk.download("vader_lexicon", quiet=True)
sia = SentimentIntensityAnalyzer()

#  Inspect and standardise
print(f"Reviews shape: {reviews_csv_df.shape}")
print(f"Columns: {reviews_csv_df.columns.tolist()}")

reviews_csv_df.columns = reviews_csv_df.columns.str.strip().str.lower()

#  Rename columns for clarity
rename_map = {"review": "review_text", "score": "rating"}
reviews_csv_df.rename(columns={k: v for k, v in rename_map.items() if k in reviews_

#  Clean review text and rating values
reviews_csv_df = reviews_csv_df.dropna(subset=["review_text"])
reviews_csv_df = reviews_csv_df[reviews_csv_df["review_text"].str.strip() != ""]

reviews_csv_df["rating"] = pd.to_numeric(reviews_csv_df["rating"], errors="coerce")
reviews_csv_df = reviews_csv_df[reviews_csv_df["rating"].between(0, 5)]

#  Add sentiment score
print("Calculating sentiment scores...")
reviews_csv_df["sentiment_score"] = reviews_csv_df["review_text"].astype(str).apply

#  Summary

print(f"Total valid reviews: {len(reviews_csv_df):,}")
print(f"Average rating: {reviews_csv_df['rating'].mean():.2f}")
print(f"Average sentiment: {reviews_csv_df['sentiment_score'].mean():.2f}")

#  Preview result
display(reviews_csv_df[["user_id", "product_id", "rating", "sentiment_score", "revi
```

```
[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
Requirement already satisfied: seaborn in c:\users\jess\appdata\local\programs\pytho
n\python313\lib\site-packages (0.13.2)
Requirement already satisfied: matplotlib in c:\users\jess\appdata\local\programs\py
thon\python313\lib\site-packages (3.10.7)
Requirement already satisfied: nltk in c:\users\jess\appdata\local\programs\python\p
ython313\lib\site-packages (3.9.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\jess\appdata\local\p
rograms\python\python313\lib\site-packages (from seaborn) (2.3.4)
Requirement already satisfied: pandas>=1.2 in c:\users\jess\appdata\local\programs\p
ython\python313\lib\site-packages (from seaborn) (2.3.3)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\jess\appdata\local\progr
ams\python\python313\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\jess\appdata\local\programs
\python\python313\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\jess\appdata\local\prog
rams\python\python313\lib\site-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\jess\appdata\local\prog
rams\python\python313\lib\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\jess\appdata\local\progra
ms\python\python313\lib\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\jess\appdata\local\programs\pyt
hon\python313\lib\site-packages (from matplotlib) (12.0.0)
Requirement already satisfied: pyparsing>=3 in c:\users\jess\appdata\local\programs
\python\python313\lib\site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\jess\appdata\local\p
rograms\python\python313\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: click in c:\users\jess\appdata\local\programs\python
\python313\lib\site-packages (from nltk) (8.3.0)
Requirement already satisfied: joblib in c:\users\jess\appdata\local\programs\python
\python313\lib\site-packages (from nltk) (1.5.2)
Requirement already satisfied: regex>=2021.8.3 in c:\users\jess\appdata\local\progra
ms\python\python313\lib\site-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in c:\users\jess\appdata\local\programs\python\p
ython313\lib\site-packages (from nltk) (4.67.1)
Requirement already satisfied: pytz>=2020.1 in c:\users\jess\appdata\local\programs
\python\python313\lib\site-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\jess\appdata\local\program
s\python\python313\lib\site-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\jess\appdata\local\programs\pyth
on\python313\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: colorama in c:\users\jess\appdata\local\programs\pyth
on\python313\lib\site-packages (from click->nltk) (0.4.6)
Reviews shape: (39063, 5)
Columns: ['product_id', 'user_id', 'rating', 'review_text', 'sentiment_score']
Calculating sentiment scores...
Total valid reviews: 39,063
Average rating: 1.49
Average sentiment: 0.67
```

	user_id	product_id	rating	sentiment_score	review_text
0	fsdv4141	b6c0b6bea69c722939585baeac73c13d	2	0.9423	You never have to worry about the fit...Alfred...
1	krpz1113	b6c0b6bea69c722939585baeac73c13d	1	0.8408	Good quality fabric. Perfect fit. Washed very ...
2	mbmg3241	b6c0b6bea69c722939585baeac73c13d	2	0.9514	I do not normally wear pants or capris that ha...
3	zeqg1222	b6c0b6bea69c722939585baeac73c13d	0	0.9329	I love these capris! They fit true to size and...
4	nvfn3212	b6c0b6bea69c722939585baeac73c13d	3	0.7172	This product is very comfortable and the fabri...
5	aajh3423	b6c0b6bea69c722939585baeac73c13d	2	-0.4973	I did not like the fabric. It is 100% polyeste...
6	usvp2142	b6c0b6bea69c722939585baeac73c13d	1	0.8402	What a great deal. Beautiful Pants. Its more t...
7	yemw3321	b6c0b6bea69c722939585baeac73c13d	1	0.9200	Alfred Dunner has great pants, good fit and ve...
8	tpcu2211	93e5272c51d8cce02597e3ce67b7ad0a	3	0.9423	You never have to worry about the fit...Alfred...
9	vutl2421	93e5272c51d8cce02597e3ce67b7ad0a	2	0.8408	Good quality fabric. Perfect fit. Washed very ...

In [13]:
```
import numpy as np
import pandas as pd
import seaborn as sns
```

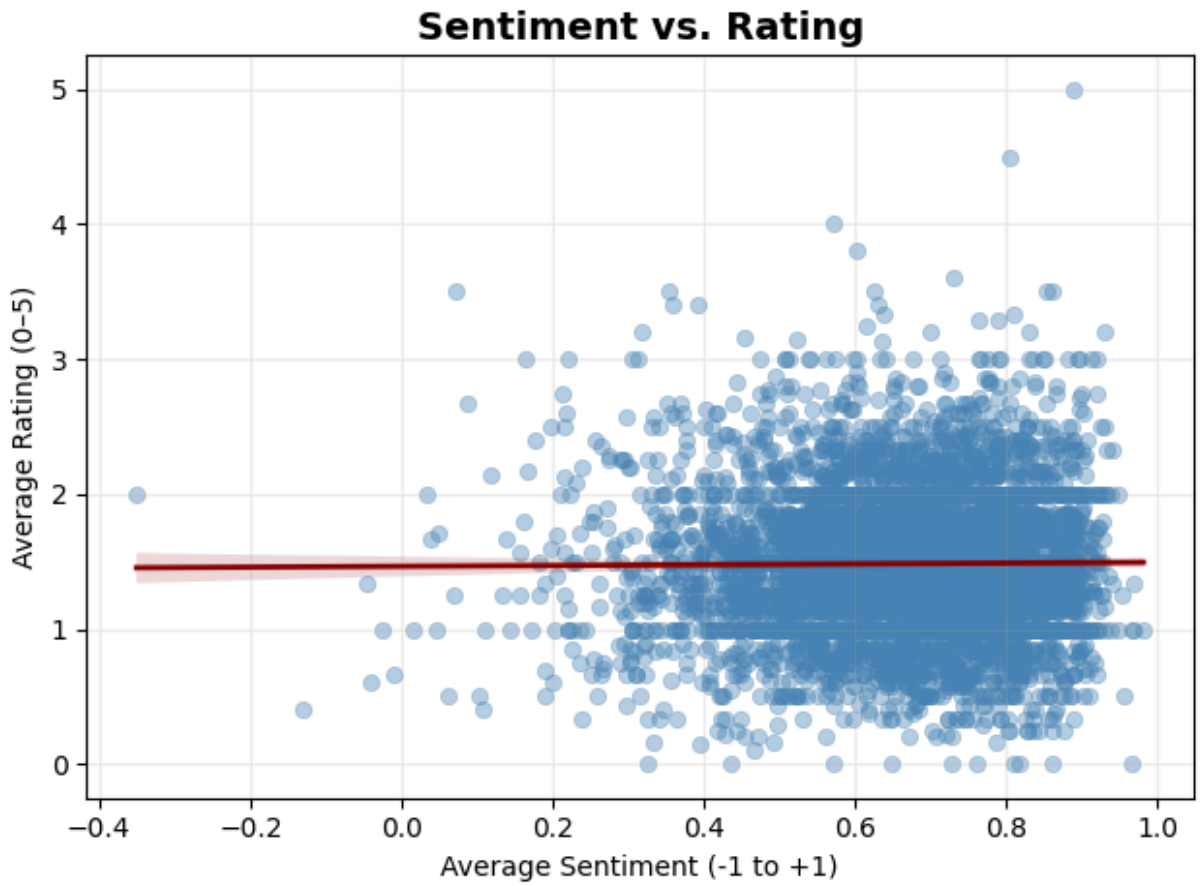
```python
import matplotlib.pyplot as plt

# Merge reviews with products
merged_reviews_products = reviews_csv_df.merge(
    products_json_df,
    on="product_id",
    how="left"
)
print(f"Merged reviews + products: {len(merged_reviews_products):,} rows")

# Merge with users
merged_all = merged_reviews_products.merge(
    users_csv_df,
    on="user_id",
    how="left"
)
print(f"Final merged dataset: {len(merged_all):,} rows")

# Create base persona dataset (aggregate user-level data)
persona_df = (
    merged_all.groupby("user_id")
    .agg({
        "rating": "mean",
        "sentiment_score": "mean",
        "age": "mean",
        "product_id": "count",
        "sale_price": "mean",
        "category": lambda x: x.mode().iloc[0] if not x.mode().empty else np.nan
    })
    .reset_index()
)

# Rename columns for clarity
persona_df.rename(columns={
    "rating": "avg_rating",
    "sentiment_score": "avg_sentiment",
    "age": "avg_age",
    "product_id": "review_count",
    "sale_price": "avg_price",
    "category": "fav_category"
}, inplace=True)


display(persona_df.head(10))

# Check for missing values
print("\nMissing values summary:")
print(persona_df.isna().sum())

# Visualise sentiment vs rating with a regression line
sns.scatterplot(
    data=persona_df,
    x="avg_sentiment",
    y="avg_rating",
    color="steelblue",
    alpha=0.4,
```

```
    edgecolor=None
)

sns.regplot(
    data=persona_df,
    x="avg_sentiment",
    y="avg_rating",
    scatter=False,
    color="darkred",
    line_kws={"linewidth": 2}
)

plt.title("Sentiment vs. Rating", fontsize=14, weight="bold")
plt.xlabel("Average Sentiment (-1 to +1)")
plt.ylabel("Average Rating (0–5)")

plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

```
Merged reviews + products: 39,063 rows
Final merged dataset: 39,080 rows
```

	user_id	avg_rating	avg_sentiment	avg_age	review_count	avg_price	fav_category
0	aaez2213	0.900000	0.799020	66.2	10	50.674000	
1	aage1132	1.300000	0.775490	67.2	10	38.440000	
2	aagf4423	0.888889	0.704544	51.3	9	82.595556	big kid (6-20)
3	aahc4142	1.000000	0.747518	58.3	11	99.902727	
4	aajh3423	1.700000	0.381460	67.2	10	37.311110	alfred dunner
5	aajn1114	1.000000	0.868950	68.2	6	29.370000	essentials
6	aakg1123	1.555556	0.774389	27.3	9	41.021111	view all
7	aaoj3233	1.500000	0.328060	74.2	10	51.222000	tops
8	aarw4144	0.888889	0.715833	73.2	9	35.596667	collared & button-front
9	aatw3332	1.500000	0.453650	57.3	4	26.292500	cotton

```
Missing values summary:
user_id          0
avg_rating       0
avg_sentiment    0
avg_age          0
review_count     0
avg_price        0
fav_category     0
dtype: int64
```



In [15]:
```
!pip install wordcloud
```

```
Requirement already satisfied: wordcloud in c:\users\jess\appdata\local\programs\pyt
hon\python313\lib\site-packages (1.9.4)
Requirement already satisfied: numpy>=1.6.1 in c:\users\jess\appdata\local\programs
\python\python313\lib\site-packages (from wordcloud) (2.3.4)
Requirement already satisfied: pillow in c:\users\jess\appdata\local\programs\python
\python313\lib\site-packages (from wordcloud) (12.0.0)
Requirement already satisfied: matplotlib in c:\users\jess\appdata\local\programs\py
thon\python313\lib\site-packages (from wordcloud) (3.10.7)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\jess\appdata\local\progr
ams\python\python313\lib\site-packages (from matplotlib->wordcloud) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\jess\appdata\local\programs
\python\python313\lib\site-packages (from matplotlib->wordcloud) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\jess\appdata\local\prog
rams\python\python313\lib\site-packages (from matplotlib->wordcloud) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\jess\appdata\local\prog
rams\python\python313\lib\site-packages (from matplotlib->wordcloud) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\jess\appdata\local\progra
ms\python\python313\lib\site-packages (from matplotlib->wordcloud) (25.0)
Requirement already satisfied: pyparsing>=3 in c:\users\jess\appdata\local\programs
\python\python313\lib\site-packages (from matplotlib->wordcloud) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\jess\appdata\local\p
rograms\python\python313\lib\site-packages (from matplotlib->wordcloud) (2.9.0.post
0)
Requirement already satisfied: six>=1.5 in c:\users\jess\appdata\local\programs\pyth
on\python313\lib\site-packages (from python-dateutil>=2.7->matplotlib->wordcloud)
(1.17.0)
```

```
[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [16]:

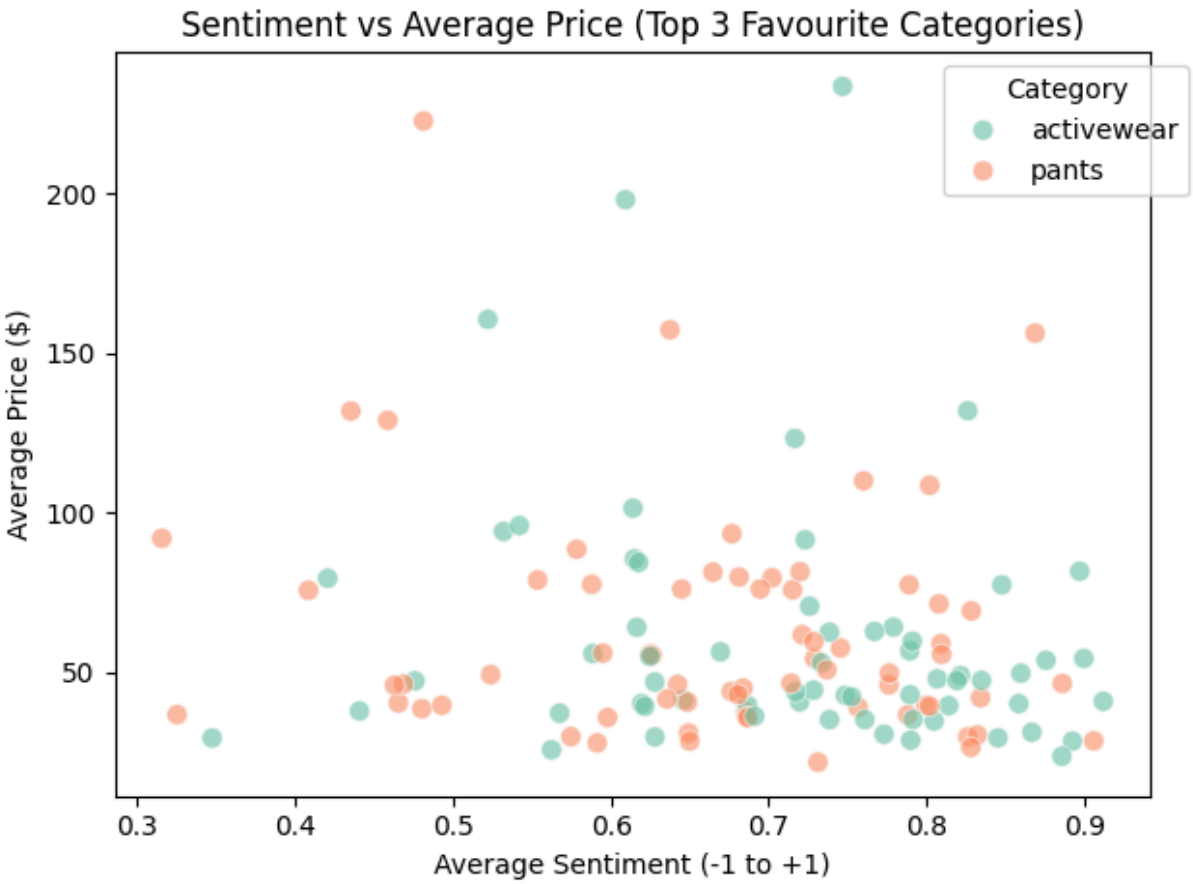
```python
# --- WORD CLOUDS (Overall, Positive, Negative) ---


from wordcloud import WordCloud, STOPWORDS
import matplotlib.pyplot as plt

# Combine and clean all review text
text_all = " ".join(reviews_csv_df['review_text'].dropna().astype(str))
text_pos = " ".join(reviews_csv_df.loc[reviews_csv_df['sentiment_score'] > 0.5, 're
text_neg = " ".join(reviews_csv_df.loc[reviews_csv_df['sentiment_score'] < 0, 'revi

# Define stopwords (to remove filler words)
extra_stop = {"dress", "buy", "bought", "size", "fit", "product", "good", "love", "
stopwords = STOPWORDS.union(extra_stop)

# Helper function to plot wordcloud
def make_wc(text, title, cmap):
    wc = WordCloud(width=1000, height=600, background_color="white", stopwords=stop
    plt.figure(figsize=(10,6))
    plt.imshow(wc, interpolation='bilinear')
    plt.axis("off")
    plt.title(title, fontsize=16, weight="bold")
    plt.show()

# Create all three
make_wc(text_all, "Most Common Words in All Reviews", "cool")
make_wc(text_pos, "Most Common Words in Positive Reviews", "Greens")
make_wc(text_neg, "Most Common Words in Negative Reviews", "Reds")
```

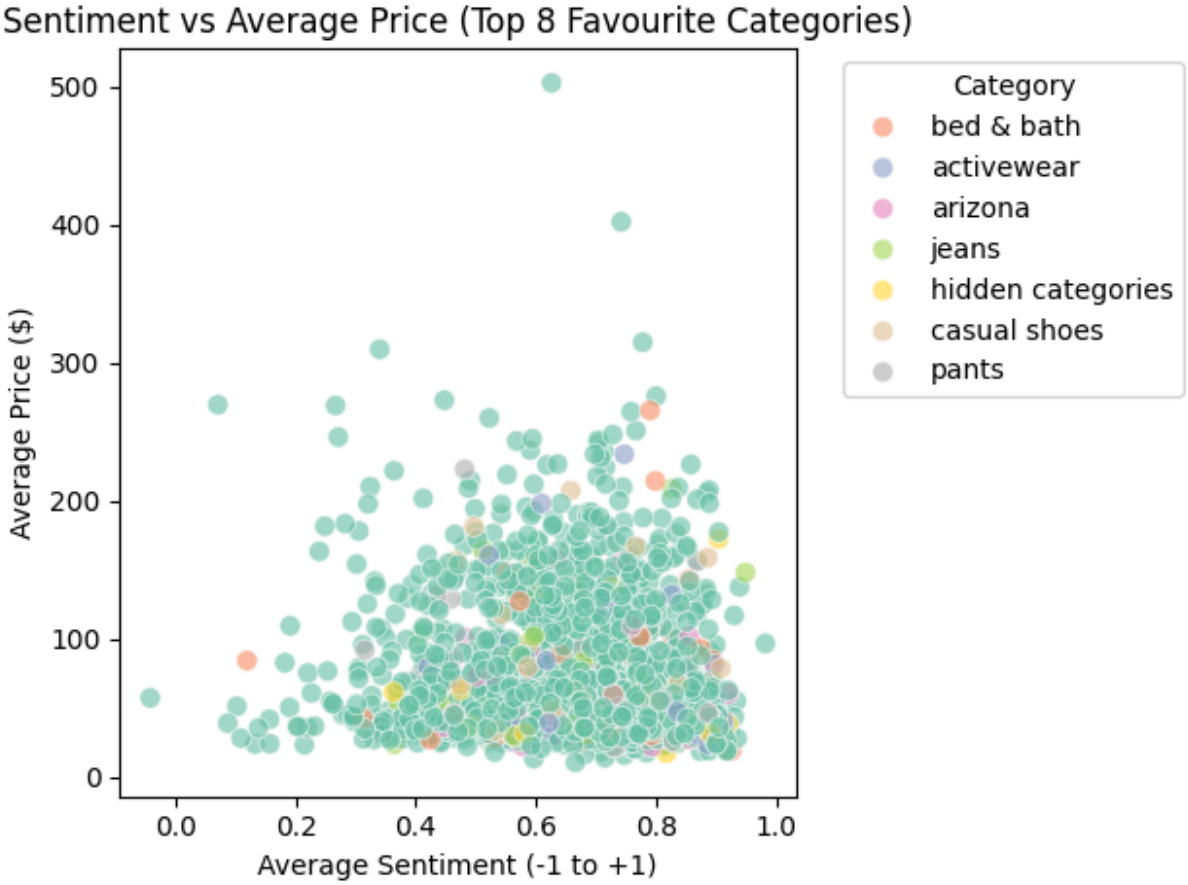
In [25]:
```
print(products_json_df.columns.tolist())
```

```
['product_id', 'sku', 'name_title', 'description', 'list_price', 'sale_price', 'cate
gory', 'category_tree', 'average_product_rating', 'product_url', 'product_image_url
s', 'brand', 'total_number_reviews', 'reviews', 'bought with', 'price_missing_flag',
'discount_flag', 'discount_percent']
```

In [34]:
```
# Relationship Exploration – Price, Sentiment, Rating
import seaborn as sns, matplotlib.pyplot as plt, numpy as np, pandas as pd
```

```
df_valid = persona_df.copy()

# Clean category names
def clean(cat):
    if pd.isna(cat): return np.nan
    cat = str(cat).lower().strip()
    bad = ['view','all','deal','clearance','offer','new','sale','gift','promo','sho
    return np.nan if any(w in cat for w in bad) else cat

df_valid['fav_category_clean'] = df_valid['fav_category'].apply(clean)
df_valid = df_valid.dropna(subset=['fav_category_clean'])

# Top 3 categories
top3 = df_valid['fav_category_clean'].value_counts().head(3).index
df_top3 = df_valid[df_valid['fav_category_clean'].isin(top3)]

df_top3 = df_top3[df_top3['fav_category_clean'].notna() & (df_top3['fav_category_cl

# Scatterplot
sns.scatterplot(
    data=df_top3,
    x='avg_sentiment',
    y='avg_price',
    hue='fav_category_clean',
    alpha=0.6, s=60, palette='Set2'
)
plt.title("Sentiment vs Average Price (Top 3 Favourite Categories)")
plt.xlabel("Average Sentiment (-1 to +1)")
plt.ylabel("Average Price ($)")
plt.legend(title='Category', bbox_to_anchor=(1.05,1))
plt.tight_layout()
plt.show()

sns.scatterplot(data=df_valid[df_valid['fav_category_clean'].isin(df_valid['fav_cat
                x='avg_sentiment', y='avg_price', hue='fav_category_clean', alpha=0
plt.title("Sentiment vs Average Price (Top 8 Favourite Categories)")
plt.xlabel("Average Sentiment (-1 to +1)")
plt.ylabel("Average Price ($)")
plt.legend(title='Category', bbox_to_anchor=(1.05,1))
plt.tight_layout()
plt.show()
```


Sentiment vs Average Price (Top 3 Favourite Categories)
Category
activewear
pants
Average Price ($)
200
150
100
50
0.3
0.4
0.5
0.6
0.7
0.8
0.9
Average Sentiment (-1 to +1)


Sentiment vs Average Price (Top 8 Favourite Categories)
Category
bed & bath
activewear
arizona
jeans
hidden categories
casual shoes
pants
Average Price ($)
500
400
300
200
100
0
0.0
0.2
0.4
0.6
0.8
1.0
Average Sentiment (-1 to +1)

In [56]:
```
import os

# 🔧 Add your MiKTeX path manually here:
miktex_path = r"C:\Users\jess\AppData\Local\Programs\MiKTeX\miktex\bin\x64"

# Add it to the system PATH for this Jupyter session
os.environ["PATH"] += os.pathsep + miktex_path

# Test whether Jupyter can now see it
!xelatex --version

import os

# Tell Jupyter where MiKTeX's xelatex.exe lives
os.environ["PATH"] += os.pathsep + r"C:\Users\jess\AppData\Local\Programs\MiKTeX\mi

# Confirm it's visible
!where xelatex
```

```
MiKTeX-XeTeX 4.15 (MiKTeX 25.4)
Â© 1994-2008 SIL International, Â© 2009-2023 Jonathan Kew, Â© 2010-2012 HÃ n Thá º¿ T
hÃ nh, Â© 2012-2013 Khaled Hosny
TeX is a trademark of the American Mathematical Society
using bzip2 version 1.0.8, 13-Jul-2019
compiled with curl version 8.4.0; using libcurl/8.4.0 Schannel
compiled with expat version 2.5; using expat_2.5.0
compiled with fontconfig version 2.13.1; using 2.13.1
compiled with freetype2 version 2.12.1; using 2.12.1
compiled with graphite2 version 1.3.14; using 1.3.14
compiled with harfbuzz version 10.1.0; using 10.1.0
compiled with icu version 76.1; using 76.1
compiled with jpeg version 9.5
compiled with liblzma version 50040002; using 50040002
compiled with libpng version 1.6.44; using 1.6.44
compiled with libressl version LibreSSL 3.8.1; using LibreSSL 3.8.1
compiled with MiKTeX Application Framework version 4.8; using 4.8
compiled with MiKTeX Core version 4.24; using 4.24
compiled with MiKTeX Archive Extractor version 4.1; using 4.1
compiled with MiKTeX Package Manager version 4.10; using 4.10
compiled with pplib version v2.05 less toxic i hope
using teckit version 2.4
compiled with uriparser version 0.9.7
compiled with zlib version 1.2.13; using 1.2.13
xelatex: major issue: So far, you have not checked for MiKTeX updates.
C:\Users\jess\AppData\Local\Programs\MiKTeX\miktex\bin\x64\xelatex.exe
```

In [98]:
```
print(merged_all.columns.tolist())
```

```
['product_id', 'user_id', 'rating', 'review_text', 'sentiment_score', 'sku', 'name_t
itle', 'description', 'list_price', 'sale_price', 'category', 'category_tree', 'aver
age_product_rating', 'product_url', 'product_image_urls', 'brand', 'total_number_rev
iews', 'reviews', 'bought with', 'price_missing_flag', 'discount_flag', 'discount_pe
rcent', 'dob', 'state', 'age', 'age_group']
```

In [125…
```
!pip install nbconvert[webpdf]
!jupyter nbconvert --to webpdf "BD2_Assignment - 2718233.ipynb" --allow-chromium-do
```

```
!jupyter nbconvert "BD2_Assignment - 2718233.ipynb" --to webpdf --allow-chromium-do
```

```
Requirement already satisfied: nbconvert[webpdf] in c:\users\jess\anaconda3\lib\site
-packages (7.16.6)
Requirement already satisfied: beautifulsoup4 in c:\users\jess\anaconda3\lib\site-pa
ckages (from nbconvert[webpdf]) (4.12.3)
Requirement already satisfied: bleach!=5.0.0 in c:\users\jess\anaconda3\lib\site-pac
kages (from bleach[css]!=5.0.0->nbconvert[webpdf]) (6.2.0)
Requirement already satisfied: defusedxml in c:\users\jess\anaconda3\lib\site-packag
es (from nbconvert[webpdf]) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in c:\users\jess\anaconda3\lib\site-packa
ges (from nbconvert[webpdf]) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in c:\users\jess\anaconda3\lib\site
-packages (from nbconvert[webpdf]) (5.7.2)
Requirement already satisfied: jupyterlab-pygments in c:\users\jess\anaconda3\lib\si
te-packages (from nbconvert[webpdf]) (0.3.0)
Requirement already satisfied: markupsafe>=2.0 in c:\users\jess\anaconda3\lib\site-p
ackages (from nbconvert[webpdf]) (3.0.2)
Requirement already satisfied: mistune<4,>=2.0.3 in c:\users\jess\anaconda3\lib\site
-packages (from nbconvert[webpdf]) (3.1.2)
Requirement already satisfied: nbclient>=0.5.0 in c:\users\jess\anaconda3\lib\site-p
ackages (from nbconvert[webpdf]) (0.10.2)
Requirement already satisfied: nbformat>=5.7 in c:\users\jess\anaconda3\lib\site-pac
kages (from nbconvert[webpdf]) (5.10.4)
Requirement already satisfied: packaging in c:\users\jess\anaconda3\lib\site-package
s (from nbconvert[webpdf]) (24.2)
Requirement already satisfied: pandocfilters>=1.4.1 in c:\users\jess\anaconda3\lib\s
ite-packages (from nbconvert[webpdf]) (1.5.0)
Requirement already satisfied: pygments>=2.4.1 in c:\users\jess\anaconda3\lib\site-p
ackages (from nbconvert[webpdf]) (2.19.1)
Requirement already satisfied: traitlets>=5.1 in c:\users\jess\anaconda3\lib\site-pa
ckages (from nbconvert[webpdf]) (5.14.3)
Requirement already satisfied: playwright in c:\users\jess\anaconda3\lib\site-packag
es (from nbconvert[webpdf]) (1.55.0)
Requirement already satisfied: webencodings in c:\users\jess\anaconda3\lib\site-pack
ages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert[webpdf]) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in c:\users\jess\anaconda3\lib\s
ite-packages (from bleach[css]!=5.0.0->nbconvert[webpdf]) (1.4.0)
Requirement already satisfied: platformdirs>=2.5 in c:\users\jess\anaconda3\lib\site
-packages (from jupyter-core>=4.7->nbconvert[webpdf]) (4.3.7)
Requirement already satisfied: pywin32>=300 in c:\users\jess\anaconda3\lib\site-pack
ages (from jupyter-core>=4.7->nbconvert[webpdf]) (308)
Requirement already satisfied: jupyter-client>=6.1.12 in c:\users\jess\anaconda3\lib
\site-packages (from nbclient>=0.5.0->nbconvert[webpdf]) (8.6.3)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\jess\anaconda3\lib
\site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert[webpdf]) (2.
9.0.post0)
Requirement already satisfied: pyzmq>=23.0 in c:\users\jess\anaconda3\lib\site-packa
ges (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert[webpdf]) (26.2.0)
Requirement already satisfied: tornado>=6.2 in c:\users\jess\anaconda3\lib\site-pack
ages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert[webpdf]) (6.5.1)
Requirement already satisfied: fastjsonschema>=2.15 in c:\users\jess\anaconda3\lib\s
ite-packages (from nbformat>=5.7->nbconvert[webpdf]) (2.20.0)
Requirement already satisfied: jsonschema>=2.6 in c:\users\jess\anaconda3\lib\site-p
ackages (from nbformat>=5.7->nbconvert[webpdf]) (4.23.0)
Requirement already satisfied: attrs>=22.2.0 in c:\users\jess\anaconda3\lib\site-pac
kages (from jsonschema>=2.6->nbformat>=5.7->nbconvert[webpdf]) (24.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in c:\users\jess
```

```
\anaconda3\lib\site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert[webpd
f]) (2023.7.1)
Requirement already satisfied: referencing>=0.28.4 in c:\users\jess\anaconda3\lib\si
te-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert[webpdf]) (0.30.2)
Requirement already satisfied: rpds-py>=0.7.1 in c:\users\jess\anaconda3\lib\site-pa
ckages (from jsonschema>=2.6->nbformat>=5.7->nbconvert[webpdf]) (0.22.3)
Requirement already satisfied: six>=1.5 in c:\users\jess\anaconda3\lib\site-packages
(from python-dateutil>=2.8.2->jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert[web
pdf]) (1.17.0)
Requirement already satisfied: soupsieve>1.2 in c:\users\jess\anaconda3\lib\site-pac
kages (from beautifulsoup4->nbconvert[webpdf]) (2.5)
Requirement already satisfied: pyee<14,>=13 in c:\users\jess\anaconda3\lib\site-pack
ages (from playwright->nbconvert[webpdf]) (13.0.0)
Requirement already satisfied: greenlet<4.0.0,>=3.1.1 in c:\users\jess\anaconda3\lib
\site-packages (from playwright->nbconvert[webpdf]) (3.1.1)
Requirement already satisfied: typing-extensions in c:\users\jess\anaconda3\lib\site
-packages (from pyee<14,>=13->playwright->nbconvert[webpdf]) (4.12.2)
```

```
[NbConvertApp] Converting notebook BD2_Assignment - 2718233.ipynb to webpdf
[NbConvertApp] WARNING | Alternative text is missing on 8 image(s).
[NbConvertApp] Building PDF
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 1488253 bytes to BD2_Assignment - 2718233.pdf
[NbConvertApp] Converting notebook BD2_Assignment - 2718233.ipynb to webpdf
[NbConvertApp] WARNING | Alternative text is missing on 8 image(s).
[NbConvertApp] Building PDF
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 1488253 bytes to BD2_Assignment - 2718233.pdf
```